Лабораторная работа 3 Версионирование данных и воспроизводимые вычисления

Версия: 2026–2027 учебный год

Формат: команда 2–5 человек с индивидуальной сдачей

Предшествующая работа: ЛР2 «Контракт данных и предотвращение утечки»

1 Цель

Связать версию проверенных данных с Git-состоянием проекта, воспроизвести две версии набора через DVC и доказать восстановление по контрольной сумме. В этой работе DVC не заменяет контракт данных: после каждого восстановления набор снова проходит валидатор ЛР2.

2 Результаты обучения

Каждый студент должен уметь:

- объяснить, почему крупные данные не хранятся непосредственно в Git;
- различать Git commit, `.dvc`-метаданные, DVC cache и remote;
- добавить разрешённый набор под контроль DVC;
- настроить локальное remote-хранилище без секретов в репозитории;
- получить и восстановить две версии данных;
- проверить SHA-256 и контракт после `dvc pull` или `dvc checkout`;
- объяснить, какие файлы нужно передавать коллеге для воспроизведения.

3 Командный результат

Команда предоставляет:

1. установленный и инициализированный DVC;
2. файл метаданных набора и корректный `.gitignore`;
3. локальное DVC remote-хранилище, путь к которому не зависит от персонального каталога участника;
4. два Git commit, соответствующие версиям данных v1 и v2;
5. успешный validation report для каждой версии;
6. протокол восстановления с командами, commit SHA, DVC hash и SHA-256 файла;
7. тест, показывающий отказ при повреждении или несогласованной версии данных;
8. issue, pull request и review другого участника;
9. индивидуальную evidence card каждого студента.

4 Границы и безопасность

Для демонстрации используется разрешённый учебный фрагмент. Полные закрытые данные, персональные сведения, токены и ключи облачных хранилищ запрещено добавлять в Git и общий DVC remote. Локальный каталог remote должен находиться вне рабочего дерева, например в каталоге, выданном преподавателем. В Git сохраняется конфигурация remote без секретов; персональные пути при необходимости помещаются в локальную конфигурацию DVC.

5 Роли

Роль	Зона ответственности	Индивидуальное доказательство
Data version owner	формирует осмысленные v1 и v2	diff данных, описание причины изменения
DVC engineer	инициализация, cache и remote	команды и diff конфигурации
Reproducibility engineer	чистое восстановление	протокол с hash и контрольными суммами
Quality reviewer	повторный запуск контракта	два validation report и отрицательный тест
Documentation reviewer	README, PR и evidence	содержательное review и исправление

При меньшем составе роли совмещаются, но у каждого остаётся собственный проверяемый вклад.

6 Подготовка

Создайте ветку и убедитесь, что ЛР2 принята:

```
git switch main
git pull --ff-only
git switch -c lab3/dvc-versioning
python -m pip install "dvc>=3,<4"
python -m pytest -q
```

Зафиксируйте версии:

```
python --version
dvc version
git rev-parse HEAD
```

7 Инициализация DVC

В корне репозитория выполните:

```
dvc init
git status --short
git add .dvc .dvcignore
git commit -m "chore: initialize DVC"
```

До commit изучите созданные файлы. Объясните, почему `.dvc/cache` не должен попадать в Git и почему служебные файлы DVC, напротив, должны версионироваться.

8 Версия данных v1

Сначала запустите контракт ЛР2 и сохраните отчёт:

```
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json --report
reports/validation/lr3-v1.json
dvc add data/flights_sample.csv
git add data/flights_sample.csv.dvc data/.gitignore reports/validation/lr3-v1.json
git commit -m "data: register flight sample v1"
```

Если политика репозитория исключает генерируемые reports, сохраните отчёт как CI artifact и укажите URI в протоколе. Запишите Git commit, DVC hash из `.dvc`-файла и SHA-256 исходного CSV.

9 Локальное remote-хранилище

Пример для каталога, созданного преподавателем рядом с репозиторием:

```
dvc remote add -d course-local ../dvc-storage
dvc push
dvc remote list
```

Не используйте абсолютный персональный путь в общей конфигурации. Если это неизбежно, задайте его через `--local` и документируйте, что коллеге нужно настроить собственный путь.

Проверьте восстановимость: удалите только рабочую копию отслеживаемого CSV предусмотренной DVC-командой или переместите её во временный каталог, затем выполните `dvc pull`. Не удаляйте каталог проекта и remote.

10 Версия данных v2

Создайте осмысленное изменение: добавьте разрешённые строки более позднего периода или исправьте документированную ошибку. Сохраняйте сортировку по `FlightDate` и не меняйте test-период задним числом без отдельного решения.

```
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json --report
reports/validation/lr3-v2.json
dvc add data/flights_sample.csv
dvc push
git add data/flights_sample.csv.dvc reports/validation/lr3-v2.json
git commit -m "data: update flight sample to v2"
```

В описании commit укажите причину, источник новых строк, изменение периода и влияние на границы train/validation/test.

11 Восстановление двух версий

Протокол выполняется из чистого клона или после безопасного сохранения незакоммиченных изменений.

Для каждой версии:

```
git switch --detach <commit-v1-or-v2>
dvc pull
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json --report
reports/validation/restored.json
```

Сравните полученный SHA-256 с записанным значением. После проверки вернитесь в рабочую ветку. Не применяйте `git reset --hard` как способ очистки: он может уничтожить незакоммиченные изменения.

12 Отказные сценарии

Проверьте минимум два сценария:

1. Рабочий CSV изменён вручную без `dvc add`: `dvc status` показывает несогласованность.
2. Восстановленная версия нарушает контракт или порядок дат: валидатор возвращает ненулевой код.

Дополнительно можно временно сделать remote недоступным и описать понятное сообщение DVC. После эксперимента восстановите конфигурацию.

13 Протокол воспроизводимости

Создайте `docs/reproducibility/lr3.md` с таблицей:

Версия	Git commit	DVC hash	SHA-256 CSV	Период	Строк	Контракт	Результат
v1	...	...	...	...	...	2.0.0	passed
v2	...	...	...	...	...	2.0.0	passed

Ни один hash не заменяет остальные: Git идентифицирует проект, DVC — объект данных, SHA-256 отчёта связывает конкретный файл с проверкой.

14 Индивидуальная сдача

Каждый участник заполняет отдельную карточку:

```
# Индивидуальная карточка ЛР3

- ФИО, группа, команда, GitHub-логин:
- роль:
- issue, pull request, commit:
- мои 2–4 изменения со ссылками на diff:
- review работы другого участника и найденная проблема:
- команда восстановления выбранной версии:
- Git commit, DVC hash и SHA-256:
- результат валидатора:
- отказной сценарий и объяснение:
- чем Git отличается от DVC cache и remote:
```

```
- вопрос преподавателя и ответ:
```

На защите преподаватель назначает v1 или v2. Студент самостоятельно восстанавливает её, запускает валидатор, показывает три идентификатора и объясняет собственный diff. Для команд из пяти человек проверяются разные действия или версии.

15 Критерии оценивания

Критерий	Баллы	Полное выполнение
DVC и структура репозитория	2	инициализация, метаданные и ignore корректны
Две версии данных	2	осмысленные v1/v2 связаны с двумя Git commit
Remote и восстановление	2	обе версии получаются из remote в чистом состоянии
Контракт и отказ	2	отчёты успешны, отрицательные сценарии воспроизводимы
Командный процесс	1	issue, PR и содержательное review
Индивидуальная защита	1	собственное восстановление, вклад и объяснение

Критические условия: отсутствие индивидуальной демонстрации, публикация запрещённых данных или невозможность восстановить назначенную версию означают, что работа не принята до исправления.

16 Чек-лист приёмки

- [ ] Команда состоит из 2–5 человек.
- [ ] В Git нет полного набора, DVC cache и секретов.
- [ ] Зафиксированы два Git commit и два DVC hash.
- [ ] Обе версии находятся в remote и восстанавливаются.
- [ ] Для обеих версий проходит контракт 2.0.0.
- [ ] SHA-256 после восстановления совпадает с протоколом.
- [ ] Есть минимум два отказных сценария.
- [ ] PR прошёл review другого участника.
- [ ] У каждого студента есть собственная evidence card и индивидуальная демонстрация.

17 Связь с ЛР4

В ЛР4 каждая MLflow run должна ссылаться на Git commit и версию данных из протокола ЛР3. Сравнение экспериментов без этой связи не считается воспроизводимым.